

Online Store Kurdi

Features Roadmap

Generated 2026-08-23 from FEATURES.md

Document scope: Every feature the project needs to be a production-ready e-commerce platform, mapped to what's actually in the repo today. Each entry has a status, the user-facing description, the implementation pointer, and the next step. **Test suite as of this document:** 712 tests, 63 files. See the "Test suite" section at the end.

How to read this document

Symbol	Meaning
Symbol	Meaning
Symbol	Meaning
Symbol	Meaning
Symbol	Meaning

Status covers **both** the API and the storefront. A "[done]" means the feature works in both halves. A "[partial]" usually means the API or the admin page has the feature but the storefront needs work (or vice versa).

Snapshot

Bucket	Total	[done]	[partial]	[missi...
Bucket	Total	[done]	[partial]	[missi...
Bucket	Total	[done]	[partial]	[missi...
Bucket	Total	[done]	[partial]	[missi...
Bucket	Total	[done]	[partial]	[missi...
Bucket	Total	[done]	[partial]	[missi...

The roughly 30 "delivered" features are subsumed under the rows below: a row marked [done] means the core flow is built and tested; a row marked [partial] means the foundation is there but the polish is missing.

Section 1: Missing features (red)

The 20 features a real production store usually has that are **not in the codebase** today. Priority order within this section: **marks the top 10 across the whole document.**

What it is: A real payment gateway (Stripe). The current `payments` module is a **mock** gated by `PAYMENTS_ALLOW MOCK`. **Why it matters:** The whole point of a store. **Where it would live:** `apps/api/` (replace mock), `apps/web/app/checkout/` (Stripe Elements or Checkout). **Effort:** S. Replace the mock with `stripe` SDK, add a webhook handler for

`payment_intent.succeeded`, switch the storefront checkout to a Stripe redirect or embedded form.

Risks: PCI scope (use Stripe Elements to keep it minimal), 3D Secure, refund flow, idempotency keys, webhook signature verification. **Test impact:** New module needs its own integration tests with Stripe's test cards; the webhook signature path needs contract tests.

Transactional emails

What it is: Order confirmations, shipping updates, password reset emails. The codebase has `order` and `contact` (form-to-inbox only) but no transactional mailer. **Why it matters:** Customers need it. **Where it would live:** `apps/api/src/utils/mailer.ts` (new), `apps/web/app/emails/`, `apps/api/src/modules/auth/`. **Effort:** S. Pick a provider (e.g. Resend), write one `send(template, to, data)` helper, template the 5 emails

that matter (order confirmation, shipped, delivered, password reset, welcome). **Risks:** Bounce handling, unsubscribe headers, DKIM/SPF, template i18n (the store already supports `ku/ar/en/tr`). **Test impact:** Unit tests for the helper, integration tests that assert the right template fires for each event.

Order PDF invoice + packing slip

What it is: Downloadable PDF invoices and printable packing slips attached to each order. **Why it matters:** Operations and legal need it; many jurisdictions require invoice retention. **Where it would live:** `apps/api/src/exports/invoice/`, `apps/web/app/orders/:id/packing-slip.pdf`, generated by `@react-pdf/renderer`. **Effort:** S. Render the existing order data into a fixed template. No

schema changes. **Risks:** Currency formatting, RTL layout (`ku/ar` invoices), large orders that need multi-page tables. **Test impact:** Snapshot tests on the PDF output, integration tests for auth (admin-only) and missing order ID.

Schema.org JSON-LD

What it is: `<script type="application/ld+json">` blocks emitting `Product`, `Review`, `BreadcrumbList`, `Organization` schema.org markup on every public page. **Why it matters:** Big SEO win. Google rich results. **Where it would live:** Each Next.js page's metadata API, or a `<StructuredData>` component from the page component. **Effort:** S. Add a `structuredData.ts` helper per page

type. The product detail page needs `Product+Offer+AggregateRating`; the listing needs `BreadcrumbList`; the home page needs `Organization`. **Risks:** Keep it valid; Google's Rich Results Test will reject malformed JSON-LD. **Test impact:** Snapshot tests on the emitted JSON.

Faceted search/filter on storefront

What it is: Multi-select category, price range, in-stock, on-sale, rating, variant attributes. **Already shipped in this commit** — see "Advanced filtering" entry below.

Product variants as a first-class concern

What it is: Sizes, colors, materials with per-variant stock, per-variant price deltas, per-variant images, swatch pickers in the UI. **Why it matters:** Most stores need sizes/colors. **Where it would live:** The `Product` schema has `ProductVariant`; the filter feature already reads its JSON `attributes` column.

What's missing is a proper **variant picker** on the product detail page, **variant-level cart line items**, **variant-level stock decrements** (currently the parent product's `quantity` is what gets touched). **Effort:** S. Add a `variantPicker` component, switch the cart/order line to a `(productId, variantId)`

key for the inventory decrement path. **Risks:** Existing order items have nullable `variantId`; migration is additive. **Test impact:** New unit tests for the picker, integration tests for variant stock decrements.

Abandoned cart email

What it is: Email customers who added items but didn't check out, ~1h after the cart went idle. **Why it matters:** High conversion rate on existing traffic. **Where it would live:** New cron-style job in `apps/api/src/cron/` (scanning `Cart` rows by `updatedAt`), template in the mailer. **Effort:** S. Once the transactional email helper exists. **Risks:** Spam complaints if too aggressive; needs an unsubscribe link and a per-user throttle.

Cookie consent + GDPR data export/delete

What it is: Banner on first visit, granular consent, "Download my data" and "Delete my account" endpoints. **Why it matters:** Legal. EU/UK users have rights; CCPA is similar for CA. **Where it would live:** `apps/web/app/cookie-consent/` (banner), `apps/api/src/modules/users/` (export + delete endpoints). **Effort:** S. The export is a SQL dump; the delete is a cascade or anonymisation.

Risks: Real deletion is irreversible — anonymise soft, hard-delete only on user request with a confirmation step.

Two-factor authentication

What it is: TOTP-based second factor for users with `role = 'admin'`. **Why it matters:** Admin accounts need it. **Where it would live:** `apps/api/src/modules/auth/` (TOTP enrollment + `apps/web/app/admin/profile/` (QR code)). **Effort:** S. `otplib` handles TOTP; the QR is just a data URL. **Risks:** Recovery flow (backup codes); lockout if a phone is lost.

Image processing pipeline

What it is: Resize on upload, serve WebP/AVIF, generate blur placeholders, strip EXIF. **Why it matters:** Performance and privacy. **Where it would live:** `apps/api/src/modules/upload/` + a worker that post-processes to a CDN. **Effort:** M. `sharp` does the work; the complexity is in the cache layer.

11 Search ranking (relevance)

Better than `contains` substring matching. Currently the `relevance` sort exists and uses a name/description/sku scorer, but it's basic. Real stores want full-text ranking with typo tolerance, place the relevance scorer in `apps/api/es/products/productFilter.service.ts`. PostgreSQL `tsvector` or Algolia/Meilisearch.

Effort: M. With PostgreSQL: add a generated `tsvector` column + GIN index. With Algolia: redirect search.

12 Gift cards / store credit

What it is: Separate from coupons. Account balance, gift card redemption at checkout, scheduled `apps/api/src/modules/gift-cards/` + `apps/api/src/modules/GiftCard` model, a balance on `User`, redemption at checkout. Requires

schema change

13 Points & Rewards program

referral bonuses, tier discounts. **Where it would live:** New `apps/api/src/modules/loyalty/`. **Effort:** L. Points engine, expiry rules, referral tracking, tier upgrade emails.

Schema changes needed

14 Carrier integration / shipping orders

or consumables. **Where it would live:** `apps/api/src/modules/subscriptions/` + Stripe Subscriptions. **Effort:** L. Requires Stripe Subscriptions and a job runner.

15 Real-time order tracking page

`/track-order` with a status timeline, carrier link, current location. The page exists but is just carrier integration. **Where it would live:** `apps/web/app/track-order/` +

carrier API clients (AfterShip, Shippo, EasyPost). **Effort:** M.

16 Multi-currency pricing

le. **Where it would live:** Product schema needs a `prices` JSON column; `apps/web/lib/settings.ts` needs a locale-aware formatter. **Effort:** M. Schema change for prices.

17 Customer reviews with photos & verified purchaser badge

What it is: Image upload on reviews, "verified buyer" tag for people who bought the product. **Where it** `apps/api/src/modules/reviews/` (extend with `images` and `verified` derived from order `id`). **Effort:** S. The review model already has `isVerified`; the image side needs the upload pipeline.

18 Product Q&A

What it is: Customers ask, staff/public answers, threaded under each product. **Where it would live:** `apps/api/src/modules/questions/`. **Effort:** M. Full model + email notifications + moderation.

19 Size guides & fit tools

; "find your size" wizard. **Where it would live:** `apps/api/src/modules/products/` (size guide table) + a wizard component. **Effort:** S for static size charts, M for the

20 Live chat / support widget

or a self-hosted chat. **Where it would live:** `apps/web/components/ChatWidget.tsx`. **Effort:** S if third-party, L if self-hosted.

Section 2 — Partial features (amber)

91. Partial Mail database but feel like scaffolding. 14 items.

`newsletter/`, `apps/api/src/modules/contact/`. **Gap:** No transactional `modules/newsletter/newsletter.routes.ts`,

`apps/api/src/modules/contact/contact.routes.ts`. **Test coverage:** 50 + 69 tests across both

modules.

92. Partial Analytics `apps/api/src/modules/analytics/`, plus a recommendations module that emits "trending" and

"new arrivals" lists. **Gap:** No GA / Plausible / Meta Pixel in the storefront. No admin-facing dashboard

`apps/api/src/modules/analytics/`

`analytics.routes.ts`. **Test coverage:** 82 tests.

93. Partial SEO `web/app/sitemap.ts`, `robots.txt`-equivalent, basic metadata in layouts. **Gap:** No

structured data (see #4 in Missing), no canonical URLs, no hreflang for the 4 supported languages. **File**

`emap.ts`, `apps/web/app/layout.tsx`. **Test coverage:** 2 tests (the seo

helper).

94. Partial Search

`chBar.tsx` (debounced, calls `/products/search`), `apps/web/app/`

`search/page.tsx`. **Gap:** No typo tolerance, no synonym matching, no relevance ranking on the server,

eg. **File pointer:** `apps/web/components/SearchBar.tsx`, `apps/`

`web/app/search/page.tsx`. **Test coverage:** 9 component tests + smoke coverage in the API contract.

95. Partial Categories

hierarchical (`parentId` self-relation), CRUD, the new multi-select filter reads them. **Gap:** No

landing pages (most stores have a hero, a featured-collection grid, a brand list, etc., at

`apps/api/src/modules/categories/category.routes.ts`. **Test**

coverage: 114 tests

96. Partial Inventory

`api/src/modules/inventory/`. **Gap:** No backorders,

ck, no auto-reorder, no `lowStock` webhook. **File pointer:** `apps/`

97. Partial Coupons

Built: Percentage / fixed / free-shipping, expiry, usage limits, per-user limits. **Gap:** No "buy X get Y", no

auto-apply rules, no stackable coupons, no first-order-only, no customer-segment targeting. UI is basic.

`apps/api/src/modules/coupons/`. **Test coverage:** 191 tests.

98. Partial Pages (CMS)

: `Page` model with status (draft/published), slug, SEO fields, `showInFooter` flag. **Gap:** No block-

based editor (admin types raw HTML today), no media library, no scheduled publishing, no preview. **File**

`apps/api/src/modules/pages/`. **Test coverage:** 109 tests.

99. Partial Theme/appearance

`themeToCssVars`, admin can change colors, the storefront injects CSS vars. The

advanced filtering uses it for the filter chip rail. **Gap:** No font upload, no logo upload, no per-section

ingle `customCss` blob. **File pointer:** `apps/web/lib/theme.tsx`. **Test coverage:**

14 tests.

100. Partial Banners

Built: Banner CRUD, image upload, position/sort, status (draft/active/archived). **Gap:** No A/B testing,

yond start/end dates. **File pointer:** `apps/api/src/modules/banners/`

101. Partial Storage

Built: Upload, list, presigned URLs (mocked in dev). **Gap:** No resize, no WebP/AVIF conversion, no

e #10 in Missing. **File pointer:** `apps/api/src/modules/storage/`.

102. Partial Wishlist

Built: Add/remove, persist per user. **Gap:** No "share wishlist by link", no price-drop notifications, no

ems. **File pointer:** `apps/api/src/modules/wishlist/`. **Test**

coverage: 128 tests

103. Partial Stock alerts

Built: Subscribe to back-in-stock for a product, admin sees list. **Gap:** No actual email is sent when

from #2. **File pointer:** `apps/api/src/modules/stock-alerts/`.

104. Partial Maintenance gate

Built: A toggle that hides the storefront when enabled. **Gap:** No per-IP bypass for admins, no

r: `apps/web/components/MaintenanceGate.tsx`.

Section 3 — Stretch / differentiator (yellow)

28. **Item that grows 10 items.**

(CSV), multi-select types, attribute filters (?attr.size=M,L&attr.color=red), price range, in-stock, on-sale, min-rating, free-text search, 9 sort

/api/products/facets endpoint that returns count buckets for the sidebar UI. **Where it**

ma.ts — the Zod schema - apps/api/

listProducts and getFacets - apps/api/src/

es.ts — GET /api/products and GET /api/products/facets - apps/

JRL !” filter codec apps/web/components/

component - apps/web/app/products/page.tsx — the storefront

page (now server-side filtered) **Test coverage:** 31 + 23 + 1 + 5 = 60 tests across the feature.

37. **Headless Commerce API** /api is RESTful already, but a true headless store has GraphQL or at minimum a richer surface for non-Next.js storefronts (mobile, kiosk, marketplace). **Effort:** L. Recommend Hasura or a PostGraphile layer over the existing Prisma schema.

38. **Mobile app PWA** Installable storefront with push notifications. The Next.js 14 codebase already supports App Router and could become a PWA with a service worker. **Effort:** L for the SW, M for the rest.

39. **B2B features** vendors, vendor payouts, vendor dashboards, vendor-product moderation. Requires a **Vendor** model and major schema work. **Effort:** XL.

40. **B2B features** Bulk pricing tiers, quote requests, net-30 invoicing, account hierarchies (parent org !’ sub-orgs !’ users).

Effort: XL

41. **Live inventory sync** With a brick-and-mortar POS or 3PL warehouse, in real time. Webhook-driven or poll-based. **Effort:** L. r-segment, or time-based sales. The schema has **compareAtPrice** for the basic case.

42. **Localization beyond kulanen/it** M. Needs a **Price** model or extension.

43. **Localization beyond kulanen/it** onary is hand-maintained in **lib/i18n.ts**. Adding a 5th language means editing code. A real i18n setup uses message catalogues (gettext, i18next with Crowdin/Locize). **Effort:** M for the switcher, ongoing cost for translation.

44. **Accessibility** Semantic landmarks, focus management, keyboard navigation, screen-reader testing, axe-core CI. The t UI is divs-with-aria, which is a start. **Effort:** M. axe-core is one **npm i** away; the rest is fixing what it finds.

45. **Performance**

Image CDN, edge caching, partial hydration, route prefetching, bundle analysis. The store will feel slow on 3G without these. **Effort:** M for the wins, L for the full pass.

Test suite 62 files all green as of this commit.

Suite	Files	Tests
Suite	Files	Tests
Suite	Files	Tests
Suite	Files	Tests
Suite	Files	Tests
Suite	Files	Tests

Highlights of the testing infrastructure:

- `apps/api/tests/helpers/mockPrisma.ts` — Map-based in-memory Prisma replacement that supports the full query surface the codebase uses. Lives in the sandbox where the real prisma
 - `apps/api/tests/integration/productFilter.smoke.test.ts` — 23 cases that hit the real
 - `apps/api/tests/integration/productFilter.live.test.ts` — replaces 15 different query strings in a single test, printing the response on mismatch. This is the test that caught the silent-coercion
 - `apps/web/lib/filterParams.contract.test.ts` — interface of the API's Zod schema used to verify the storefront's URL encoder produces a query string the API parser accepts
- npm test

Open tradeoffs (decisions made under constraints):

1. **Saved filter presets are client-side (localStorage).** The advanced filtering feature lets the user save a filter to revisit later, but we chose localStorage over a server endpoint to avoid schema changes. Presets should sync across devices, add a `SavedFilter` model.
1. **The mock prisma powers every integration test.** The real Prisma engine can't be downloaded in this sandbox. When it becomes available, the integration tests work unchanged; `database', ...) from apps/api/tests/setup-integration.ts`.
1. **Search sort is a simple JS scorer.** Production-quality search needs PostgreSQL or a dedicated search engine. See feature #11.
1. **Filtering happens in JS after the DB query.** The `attributes` column is a JSON object. Exact-value matching can't be expressed in Prisma's `where`. This is fine for small catalogs; for 10k+ products, normalise attributes into a side table.
1. **Payments allow mock.** Before production, integrate Stripe (feature #1).
1. **Changes were made for this filtering feature.** We used existing models (`Product`, `int`, `Category`, `Review`). Adding a `SavedFilter`, `Price`, or `Tag` model would unlock more features but is out of scope for "no schema changes".

[illegible][illegible]

Appendix B — Top-10 priority for the next sprint

#	Feature	Bucket	Effort	Why now
#	Feature	Bucket	Effort	Why now
#	Feature	Bucket	Effort	Why now
#	Feature	Bucket	Effort	Why now
#	Feature	Bucket	Effort	Why now
#	Feature	Bucket	Effort	Why now
#	Feature	Bucket	Effort	Why now
#	Feature	Bucket	Effort	Why now
#	Feature	Bucket	Effort	Why now
#	Feature	Bucket	Effort	Why now
#	Feature	Bucket	Effort	Why now